ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

Corso di Laurea Magistrale in Digital Transformation Management

Fancy Title

Tesi di laurea in:
Business Intelligence

Relatore
Matteo Golfarelli

Candidato
Andrea Giovanardi

II Sessione di Laurea
Anno Accademico 2025-2026

Abstract

Max 2000 characters, strict.

Optional. Max a few lines.

Contents

List of Figures

List of Listings

Chapter 1

Introduction

1.1 Context

During one of the audited weeks of my internship, the operational dashboard flagged a clear gap at the station. The AI summary built on top of it, an early version of the system this thesis describes, explained that difference with full confidence, but the explanation produced was factually incorrect. It pointed to a structural metric that had little to do with the real cause of the difference, proposed corrective actions and root analyses that no manager could realistically carry out. I noticed the error, that morning, because I was familiar with the station and the metric, but a manager reading the same paragraph on a busy day, or a less experienced one, could have easily taken that for true and acted on it.

That episode marked the beginning of my thesis and frames the general problem it addresses: what happens when a Large Language Model (LLM) is placed on top of a BI dashboard and asked to turn numbers into decisions? That is a key question considering the setting of a Delivery Station (DS). It is the final node of the logistic network, the place where packages are sorted in the early morning and loaded onto routes to be delivered to final customers. The station where I worked, DFV2 in Udine, runs on a standard daily rhythm: plan the day, run it, measure what happens, adjust for tomorrow. The manager controls the rhythm through a large set of indicators that are read every day to determine if any correction is

needed. Normally the window to act is narrow so it is important that a manager does not get the picture late or even wrong, otherwise the chances to fix it would be lost.

Among all the indicators that we used at the station, one metric acts as the headline indicator, a single number that captures how the station operates. Most of the days this number is close to the planned one, but not exactly on it, and that gap is what a manager has to explore, understand and explain. A process that is anything but trivial. This difference is the combined effect of many separate factors: some of them are internal to the station while the rest of them are completely external, and working out which factor and its weight is a task that, before this project, was done completely by hand.

When a station misses its target, the designated manager has to produce a written justification, internally referred to as the "bridge", based on a weekly spreadsheet report that states how the previous one had closed. A manual and slow activity that said nothing about how the current week was going while there was still time to act on it.

This is the exact scenario where a LLM could be the right fit. A model capable of reading the numbers and turning them into paragraphs within seconds, replacing hours of manual assembly. The downside is that the same flexibility that makes it so powerful and useful is also the source of risks. A language model hallucinates on numbers, quietly recomputing a value or inventing one that looks plausible, and it explains causes with the same fluency whether the cause is real or not. For a tool that is used to feed decisions on the floor, those errors are not acceptable, and they must be reduced as much as possible, while keeping in mind that hallucinations cannot be avoided at 100%. A manager must be able to safely rely on the output of the model while at the same time having the underlying numbers within reach.

1.2 Criticalities

The main difficulty of this project is the grounding problem, understood in a stricter sense than the one commonly used in the literature. It can be decomposed

into two main requirements that need to hold together. The first is numerical: the number must correspond exactly to the value behind the dashboard, not an approximation of it. The second one is causal: every explanation and every recommendation must come from a well-defined set of causes that the domain admits, not from everything that seems reasonable. Figure 1.1 shows the two requirements and the related failure points.

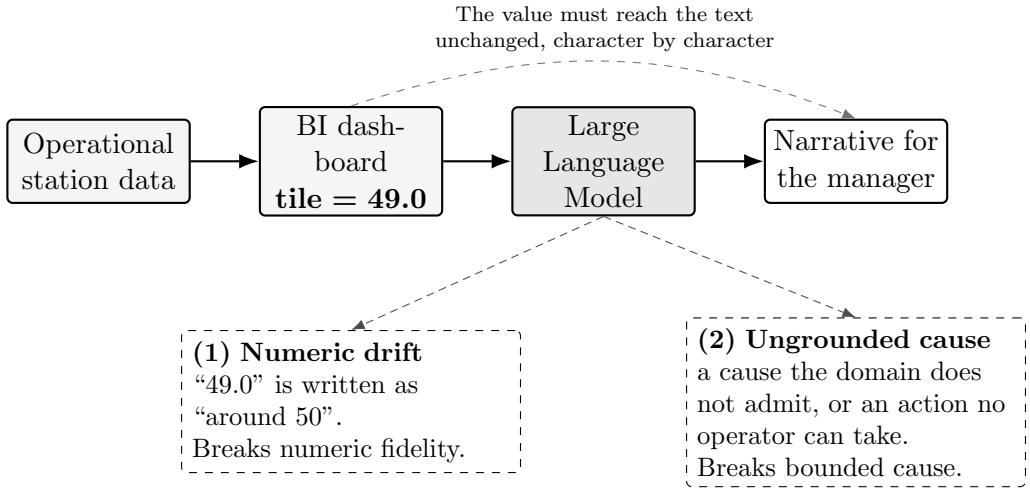

Figure 1.1: From dashboard value to narrative, and the two points where grounding breaks: a number that drifts (1) and a cause the domain does not admit (2).

Consider the numerical side first. The one that is shown to the user must be equal to the retrieved one. If that value is close but not exact, a decision taken on the basis of it can still be wrong. This is more demanding than the usual faithfulness a retrieval system offers, where a claim counts as grounded if it is broadly supported by some documentation. For Operational BI a broad support is not enough.

The causal side is even harder. Reporting a number without changing its value is a matter of staying true to the base data. Explaining why we have that precise value, instead, is connected to the domain knowledge. The set of plausible-sounding causes for any gap is enormous, while the possible set of causes admitted by the domain is small and specific. A narrative defined as "useful" has to stay inside this small set. It should behave like an expert analyst that among the possible explanations of the gap, chooses only the feasible ones, and consequently the actions that a manager can adopt in order to improve the situation. Domain

constraint without numerical fidelity buys very little, a right explanation based on a wrong input is meaningless.

Those two requirements matter even more because a wrong output does not stay only on the screen. A manager uses it as the basis for decisions that influence the whole operation. The standard here is therefore demanding, above all on the numbers: a value must be right every time, because one confident mistake, acted upon, can undo a week of planning. The causal side is a different story: the most I can require is that the explanation stays inside what the domain admits. In particular, once the numeric side was frozen upstream, most of the failures I observed were causal: a wrong number is worse, but in the running system wrong causes were what remained.

1.3 Research questions

The two criticalities highlighted in the previous section shape the questions that guide the thesis. I framed the work around one main question and two subsequent ones, each at a different level.

The first question is empirical and comes before thinking about any solution: what are the common patterns of grounding failure that occur when a language model generates a narrative over a high-density BI dashboard? I wanted a clear picture of how the model fails derived from the field directly, rather than borrowed from the literature. This decision was motivated by the intention to build a design tailored to the real failures.

From the answer to this primary question two design questions follow. The second asks what architecture can be implemented to constrain the faithfulness of the model towards both the numerical and causal sides at once, rather than patching each slip in isolation. The last question is narrower: what role can a curated knowledge tree, a structured map of causes and feasible actions, play as the base structure that bounds what the model is allowed to say?

Those three questions mark the trace that the next chapters will follow. The primary is answered by a catalog of failure modes derived from the internship

itself, the second and third by the framework that has been built to mitigate those errors and tested against the same catalog.

Structure of the Thesis The remainder of this document is organized as follows:

- **Chapter 2** reviews the state of the art and related work, exploring current paradigms for LLM integration, evaluating hallucination metrics, and positioning the work with respect to current methodologies.
- **Chapter 3** introduces the case study, detailing the operational setting, the core metric, the pre-AI baseline, and the single-case methodology adopted during the internship.
- **Chapter 4** presents the proposed solution, opening with the conceptual innovation of the three-contract framework (tile-as-source, curated knowledge tree, and sanitizer), the system architecture, and the catalog of grounding failures.
- **Chapter 5** focuses on the results and evaluation, measuring the framework against failure modes and comparing the generated narratives with the bridges managers wrote by hand.
- **Chapter 6** discusses the findings, highlighting the thesis contributions and generalizability, acknowledging limitations, and outlining future research directions.

Chapter 2

State of the Art and Related Work

Chapter 1 framed the problem as two requirements that have to hold together. Every number that appears in the generated text must correspond to its actual value and every explanation must come from a well-defined domain. This chapter focuses on the existing literature through that double lens. For each family of methods, I run the same two checks: does it guarantee the truthfulness of the number, and does it bound the cause? The map this builds is what the last section uses to present the work.

2.1 The grounding problem

When we talk about grounding, in terms of natural language processing, we usually mean that the output is supported by the source. Retrieval-Augmented Generation (RAG) made this idea popular: a claim counts as grounded when retrieved evidence backs it [1]. There are two main failure modes as stated by the survey on hallucination in text generation [2]. Intrinsic hallucination is text that contradicts the source, and extrinsic hallucination is text that adds something the source cannot confirm. The same literature goes further by separating faithfulness as being coherent with the input given, from factuality intended as the truth in the world

[3]. A text can be faithful but not factual if the input is wrong, but the narrative reports it coherently. At the same time, a text can be factual but not faithful if it states something right in the world, but is not coherent with the input. The requirement that matters for BI is faithfulness.

Operational BI requires a higher standard. Generic support is not enough. A figure of 49.0 is either reproduced exactly or it is wrong. Any representation that deviates from the defined value is incorrect. As anticipated in the introduction: a narrative is grounded when every quantitative token is traceable, character by character, to a structured field, and every comparative statement agrees in sign with the source. This aligns more with faithfulness than with factuality, and it is tighter than both because it asks for fidelity to an exact value and not for consistency with a passage. Controlled table-to-text work, the ToTTo dataset for instance [4], pushes the generation in that direction, by asking the model to describe only the given cells without any deviation.

The causal half has no real equivalent in the standard taxonomy. A phrase can have all the right numbers, but still name a cause that does not exist or propose an impossible solution. The standard question when talking about hallucinations is the following: "is the claim supported?", and not: "does this cause belong to the domain-defined set?" The second question is the dividing line between BI and open-domain generation, and in fact it is the reason why a single definition of grounding is not enough.

2.2 Retrieval-Augmented Generation (RAG)

RAG conditions the model by retrieving passages from a corpus of documents and concatenating them as context into the model's input [1]. It reduces extrinsic hallucinations, since now the model has something to lean on without inventing. It has become a common way to keep a generator close to a body of documents. The evaluation framework follows the same logic: RAGAS scores faithfulness, defined as the degree to which a claim is entailed by the retrieved context [5]. Does the generated phrase follow from the passages? If yes, it is considered grounded.

Here the unit of grounding is the passage, and the test is semantic support. For a textual response this is reasonable, but not for a number. A passage that mentions a figure "in the region of fifty" entails a sentence that says "around fifty", but that sentence fails the operational test I set above. Retrieval works by similarity, not by schema identity. RAG has no concept of canonical identity, so it cannot distinguish between an authoritative value and one that resembles it. RAG is necessary, but not sufficient, since it has no guarantee about the value.

2.3 Prompt engineering and reasoning

A lot of research focuses on the prompt to improve the output, shifting the attention to how something is asked rather than the model itself. Chain-of-thought prompting asks the model to reason by decomposing the work into steps [6]. Self-consistency creates many reasoning paths and keeps the most voted one [7]. Both methods lift the average quality of what comes out, not the single case.

None of them enforces anything. They simply make the correct answer more likely, but they do not make the wrong one impossible. As reported later, even a capable model with a competent but generic prompt tends to invent numbers or causes. Good instructions alone cannot close this gap. When sending a message to an LLM we are making a request, not a command, and since the model itself is a probabilistic generator, it is free to follow our directions or not. On the causal side, prompt engineering helps: it makes the model reason in a cleaner way, and produce fewer off-topic causes. On the operational side it cannot supply the hard guarantee that this use needs, since well-done prompt engineering only pushes the odds; no amount of it can make the value certain.

2.4 Tool use and program-aided generation

A different family gives the model access to external tools. Reasoning + Acting (ReAct) represents an iterative cycle composed of thought, action and observation. It introduces the possibility of performing a search call [8]. Toolformer is a similar

approach, capable of deciding on its own when to call an Application Programming Interface (API) [9]. Program-Aided Language (PAL) models take a step forward by generating the code that automatically performs an operation, instead of guessing it [10]. Consequently, the model avoids the hallucination risk associated with computation. In particular, this last idea is close to the one I adopted: taking the computation out of the model's hands.

All those methods share one limit. They do a great job at securing the data, meaning the input will likely be correct, but they let the model build on this information without any control or boundary. Fetching the right value is only half of the job; the other half involves representing it correctly. Nothing in these techniques checks that the number that comes out in the sentence is the same as the one that went into the prompt. The tool fixes the input. The output is still free, and a free output can still drift.

2.5 Constrained and guided generation

When we talk about constrained decoding, we refer to a family of methods that force the output into a defined shape independently of the content. Language Model Query Language (LMQL) treats the prompt not as plain text but as a small program characterized by a set of constraints [11]. A typical obligation can be: the response must be shorter than twenty tokens, and the system code will enforce that. Parsing Incrementally for Constrained Auto-Regressive Decoding (PICARD) brings this idea to Structured Query Language (SQL). It is an approach that, by parsing incrementally at each generation step, rejects any token that would make the query invalid, based on the applied constraints [12].

Constrained decoding is the closest approach to the sanitizer I implemented and described in Chapter 4. This is motivated by the fact that both of them use code to enforce, before or after the generation, a rule on the output. When talking about PICARD and LMQL, however, they focus only on the form level: a syntactically valid query or well-formed number where it belongs, but they say nothing about the correspondence of the value to the source or the free-text cause belonging to

the domain.

2.6 Semantic layers and knowledge graphs

There are two ideas from the data-engineering world that are close to what I need. The first one is the semantic layer, used by tools like data build tool (dbt) and Cube, in which the definition of a metric appears only in a single governed place [13, 14]. The core principle is a single source of truth: the value is computed in one place and read everywhere else, so two reports cannot disagree on the calculation by silently building their own version of the metric. This methodology is the one that I adopt for numbers: take a figure from a single upstream source and copy it, never letting the model work it out. However, that approach is limited because the semantic layer stops before the narrative. It returns clean and canonical numbers, but it does not say why that number moved or list, among the possible causes, the one worth reporting.

The second idea regards the causal side. Grounding an LLM in a knowledge graph is one of the most effective ways to keep what the model says inside a fixed domain. How the two can be joined is, by now, well mapped [15]: a graph defines what the domain treats as true and guides the model's reasoning, preventing it from relying on whatever has been absorbed during training. What matters most for my case is the Reasoning on Graphs (RoG). It "bounds" every step in the generation process to a specific path in the graph, such that a response can be traced back to a chain of specific relations rather than to a plausible-sounding guess [16]. Other strategies directly feed a smaller portion of the graph into the prompt itself [17], but independently of the methodology that is chosen, the key idea is to give the model a structure and let that structure decide what counts as admissible to say.

Neither family does the whole job alone. The semantic layer returns the right number but no story; knowledge-graph grounding bounds the story, but says nothing about the value. What is needed is a combination of both, together with the downstream check seen above, to make sure that the framework is capable of supporting a correct operational narrative, and it is the one I build in Chapter 4.

2.7 Generative BI and conversational analytics

The applied side of turning dashboards into narrative goes under different names: conversational BI, augmented analytics, generative BI. Its oldest thread is text-to-SQL, where a natural language question is translated into an executable query, scored by benchmarks like Spider that evaluate how accurate the transformation is [18]. A parallel line turns the same question into a chart [19]. Two different outputs originate from the same idea: a table or a picture, nothing more. Any explanation that a manager needs is outside their scope.

It has been shown that letting a model handle the numbers is risky [20]. LLMs are unreliable even on simple quantitative reasoning, because of their nature, as stated in Section 2.3. A study about text-to-SQL on real enterprise data shows semantic hallucinations with no domain-specific way to track them down [21]. This is the empirical reason behind the strict rule implemented in Chapter 4: the safest number is the one the model never computes.

Over time, commercial tools have moved closer to the narrative side. Tableau Pulse [22] and Copilot in Power BI [23] write short insights over tiles, and they are the industry baseline for this work. The most advanced of them documents an approach close to the one that I adopt: Tableau Pulse computes its metric findings with a statistical engine and uses the LLM only to phrase them [24, 25]. Templates and in-context examples reduce the risk, but there is no deterministic check on the printed value, and none of those tools publishes a reproducible measurement of how that grounding holds up practically.

The academic field has grown fast on this topic during the last year. Some systems detect the domain on their own without any human direction and elaborate multi-perspective narratives without any predefined schema or concepts [26]. Others, especially in finance, join causal-discovery statistical algorithms with budgetary variance to automate root cause diagnosis [27]. This is the case closest to mine in terms of goal, since it also tries to explain a gap.

Another approach works on the same structure of data as I do. Paradigms like Conversational and LLM-assisted Online Analytical Processing (OLAP) bring natural

language to the multidimensional cube. Normally a query is used to retrieve the information. Now the system is not limited to the conversion of text to a query, but also describes what comes back. One such system is Vocalizing OLAP (VOOL): a modular insight-based framework that extracts selected quantitative insights and turns them into a description [28]. It does not use every insight: VOOL picks only the relevant ones and only those become part of the narrative. In the field of multidimensional cubes, this last approach is the closest point to a real narrative over structured analytical data. It stays accurate by building descriptions from statistics and templates, without interpreting the results. The numbers are safe, but the reason behind them is left to the reader. Then the wall again. What happens when an LLM is put inside the loop to carry out the analysis? It can hallucinate on numbers. One of the most recent attempts, a system for LLM-assisted navigation, keeps the language model far from figures. The model does only two things: it ranks the candidate views, and it explains why. A separate OLAP engine then computes every value in order to preserve number fidelity [29].

These are the nearest neighbors of the present work, and they differ from it in two aspects: their causes are generated freely or discovered statistically, not bound to any curated knowledge tree, and they implement numerical fidelity by keeping the numbers away from the model, not by a deterministic contract over the generated narrative.

2.8 Evaluating hallucination and faithfulness

The way faithfulness is measured decides what is counted as solved. A critical part, in fact, is choosing the right approach to the scoring, one that avoids marking something as “addressed” that is not. FActScore is the most fine-grained approach: it works by breaking a long text into smaller fragments called “atomic facts” which are single elementary claims, and by calculating the percentage of those that are supported by the source [30]. HaluEval [31] and TruthfulQA [32], instead, build benchmarks of hallucinated or misleading content. A benchmark is a fixed and shared dataset of test cases that lets different systems be compared on the same ground. SelfCheckGPT goes for a different approach: with no need for any external

source, it exploits the model itself by running the same request several times. The fundamental idea behind this sampling-based check is: the model is asked for the same thing repeatedly, if the answer is fabricated, it drifts since the LLM will invent slightly different versions each time; if the multiple answers agree with each other it is considered a real fact [33].

All those instruments were not built for operational BI. For example, an atomic-fact check has no notion of the exact value, while the sampling-based detector is probabilistic: the result is not reproducible since a run is not deterministic. A retrieval faithfulness score measures support, not identity [5]. And more to the point, none of them distinguishes between a numerical error and a causal one, which is the very distinction this work is built around. For that reason, Chapter 5 measures the presence rate of each failure mode, compared across all the configurations, so the effectiveness of each contract can be read directly.

2.9 Positioning of this work

Together, all the families above show a pattern. Each of them secures at most one of the two requirements: numerical or causal. Prompting improves how the model reasons but, while it reduces domain disconnections, it is a request not a command. Tool use works well for grounding numbers, but the narrative is left free. The part related to guided decoding is stricter on form, but it does not mention the correctness of the value or the cause. The semantic layer hands the official number and stops there; knowledge-graphs bound the explanation, but say nothing about the real value.

The applied systems capable of building a narrative fall into three groups. Tableau Pulse is capable of writing insights over tiles; it describes the fidelity mechanism without any measure of it, and on the causal side it surfaces statistical drivers without binding them to a domain. A newer group of academic systems produces a real day-after narrative: one of these approaches detects the domain on its own, while another identifies the root cause by performing a statistical analysis, but in both the numbers generated by the model are not guaranteed. OLAP-narrative

systems like VOOL, instead, focus on the number's fidelity by keeping the LLM away from them, yet they stop at description or navigation, and none of them ties the cause to a curated set. With all of them the day-after explanation, in the end, rests on trust and not on a check.

Table 2.1 summarizes all those approaches. The two middle columns contain the requirements from Chapter 1, and the last one asks whether the approach produces a written justification. In particular, in the last row a yes appears in both requirements, but the two carry different meanings. The numeric one is hard because every value the contract covers is a copy of the upstream one and a deterministic check guarantees that; where that promise stops is declared in Chapter 4. The causal one is soft: a knowledge tree defines the domain, but there is no enforcement of that as it happens on the value's side.

Approach	**Exact number**	**Bounded cause**	**Day-after**	
RAG	no	no	no	*techniques*
Prompt engineering	no	no	no	
Tool use	partial	no	no	
Constrained decoding	no	no	no	
Semantic layers	yes	no	no	
Knowledge graphs	no	partial	no	
Text-to-SQL / NL2VIS	partial	no	no	
Commercial BI assistants	partial	partial	yes	*narrative*
LLM-generated narratives	no	partial	yes	
LLM-assisted OLAP	yes	no	partial	
This thesis	**yes** (hard)	**yes** (soft)	**yes**	

Table 2.1: Existing approaches against the two requirements of operational BI narrative, and whether they produce a day-after explanation at all.

Figure 2.1 represents the same idea on the two requirements alone. Numerical fidelity is located on the x-axis, causal grounding on the y-axis, and each of them has three possible states: none, partial, full. This work sits in the top-right corner, while every other family lands off a side.

As presented, across the ten families reviewed, not one fully addresses both require-

ments while also writing the explanation a manager reads the day after. There is a reason for it: the two sides are not the same kind of problem. The number can be made a hard guarantee, while the cause only bounded to what the domain admits. The chapters that follow go in that direction, exploring the case where the gap first appeared.

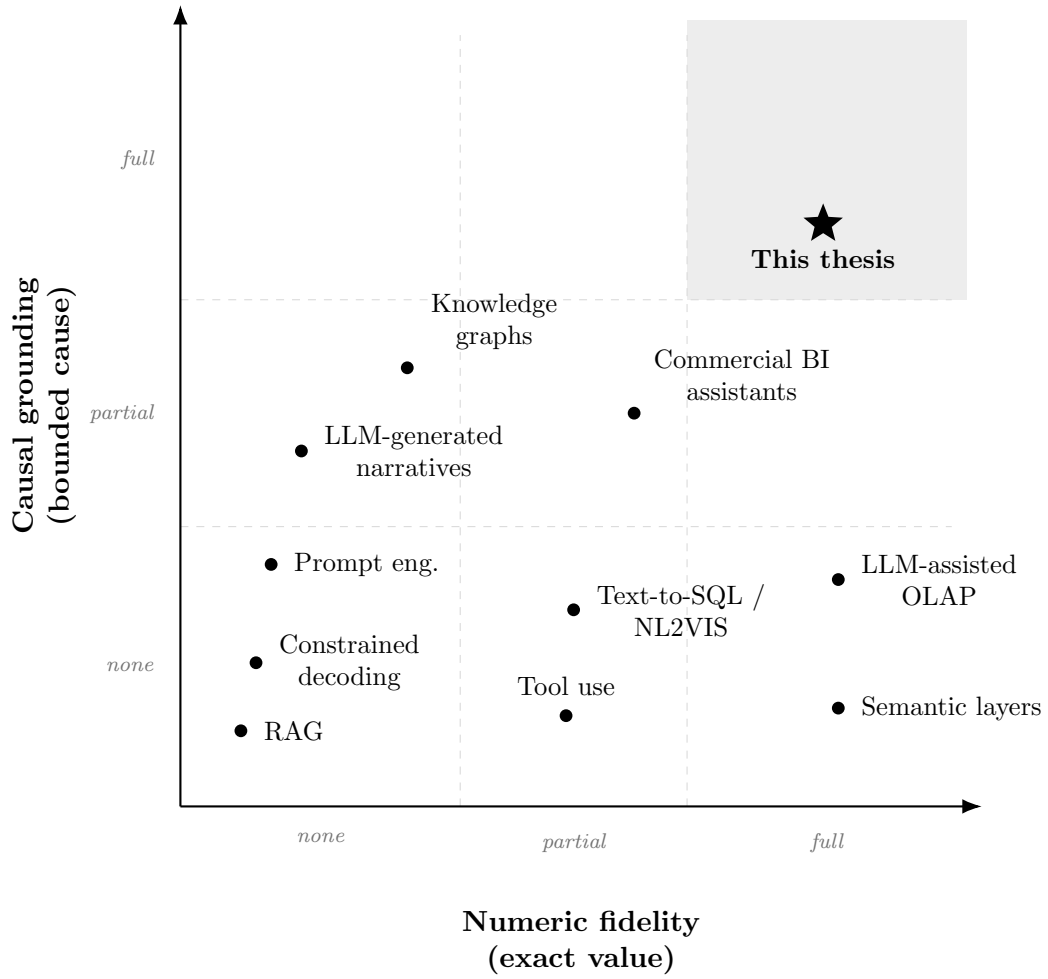

Figure 2.1: The reviewed approaches on the two requirements of operational BI: numeric fidelity (horizontal) and causal grounding (vertical). The star marks this work; on the causal axis the guarantee is bounded rather than verified (soft, see Chapter 4).

Chapter 3

The industrial case study

This chapter covers the foundations of my case study. It is about the operational setting, the headline indicator the weekly review turns on, and what builds it up. Then come the manual practices done before any automated support, and the related weaknesses. The methodology closes the chapter: development, data selection and confidentiality.

3.1 Operational setting

The setting is the one introduced in Chapter 1: I carried out my internship at DFV2, a last-mile delivery station located in Udine. The focus of my work is not the station as a whole, but a specific weekly task carried out by managers across Europe. The numbers are watched daily, at the cost of pulling them by hand, while the gap explanation is prepared only once a week. In my case, this explanation does not stay local: it goes up to a higher team that oversees the whole country and looks at every DS that missed its target.

3.2 The headline indicator and its decomposition

The headline indicator can be defined as output quantity over resources used to produce it, in simple words a ratio. The difference between what was planned and the realized value is the gap that my project focuses on. This indicator is then decomposed into a pre-defined and fixed set of categories that are mutually exclusive. The set is organized on two levels: top-level categories that open into sub-factors (Figure 3.1). This second level is needed because it shows which specific part caused the shift.

The categories do not contribute to the total gap as a raw value, but they get scaled by their own weight, which is heterogeneous and changes across all categories. Each category has its own unit of measure; the weight translates a movement in that unit into points of the headline ratio that can now be summed up since they speak the same language. This means that a large movement in a lightly-weighted category can matter less than a smaller movement in a heavy one. There is also a residual term that absorbs whatever difference the named categories do not explain. It guarantees that the parts add back to the whole, and prevents anything from being forced into a named category where it does not belong.

Table 3.1 makes clear why the gap must be decomposed: the categories offset each other, some of them push the difference up while others pull it down. A category that helps can hide a problematic one, so the net figure on its own is a poor guide to where the problems are located. Before my project, this whole reading was done manually.

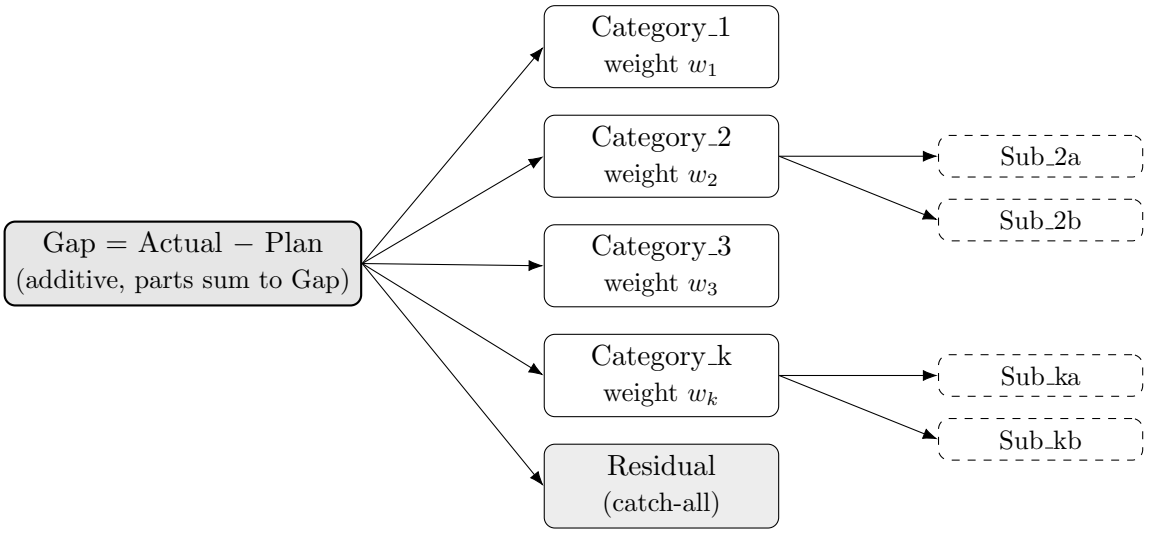

Figure 3.1: Two-level additive decomposition of the gap (anonymized). Top-level categories are mutually exclusive; selected categories open into sub-factors; a residual term closes the sum.

Component	**Raw movement**	**Weight** w_i	**Contribution** (= mov. × w_i)
Category_1	+1.50	0.30	+0.45
Category_2	−2.00	0.40	−0.80
of which Sub_2a			−0.55
of which Sub_2b			−0.25
Category_3	−1.20	0.50	−0.60
Category_k	+1.00	0.10	+0.10
of which Sub_ka			+0.18
of which Sub_kb			−0.08
Residual			−0.35
Total (Gap)			**−1.20**

Table 3.1: Illustrative numeric example of the gap decomposition with synthetic values and weights.

3.3 The manual baseline and its weaknesses

The starting baseline involved two main time-consuming practices: monitoring and writing the explanation. A manager who wanted to know how the running week was going had to open one dashboard at a time and copy-paste all the data needed into a new spreadsheet file used for the analysis. Each of those dashboards is a grid of precomputed values that a manager reads as they are: each of those values is a tile. All the data sat in several places, in a tabular way with no graphs or visualization that could help the reader. The second activity was the production of the bridge document that explains the gap: what caused it, why, and actions taken in response. A bridge is not a complex report, but a compact paragraph that is characterized by the same three ingredients: numbers, causes and actions. The causes a manager identifies as responsible for the gap are backed by the numbers pasted from the spreadsheet, and paired with the feasible actions the station will take to improve the situation. Those ingredients, together with the completeness of the drivers, are the four axes the generated narrative is scored on in Chapter 5.

That manual approach has three different weaknesses starting from limited reactivity: a manager had to wait for the weekly report to know how the previous week had closed exactly; any intra-week visibility, when needed, depended on pulling data manually, tile by tile. The second one is incomplete root-cause coverage: the choice of which causes needed to be added to the report was up to the manager, who would typically scan the spreadsheet for KPIs sitting below target and write them up as the explanation. The decomposition existed in the data, but nothing forced the written explanation to follow it: no guarantee that the listed causes summed back to the total gap, nor that they were ranked by impact. The last one is analyst dependence: the depth and granularity of the diagnosis, and the realism of the recommended actions, varied considerably with the individual experience of the manager preparing the document. The three weaknesses are visible in Figure 3.2 together with a description of how each one is answered.

The three weaknesses do not all sit on the same level. On the causal side, the causes the manager wrote were often correct but rarely complete, with smaller contributors sometimes not included: the manual problem is one of coverage, not

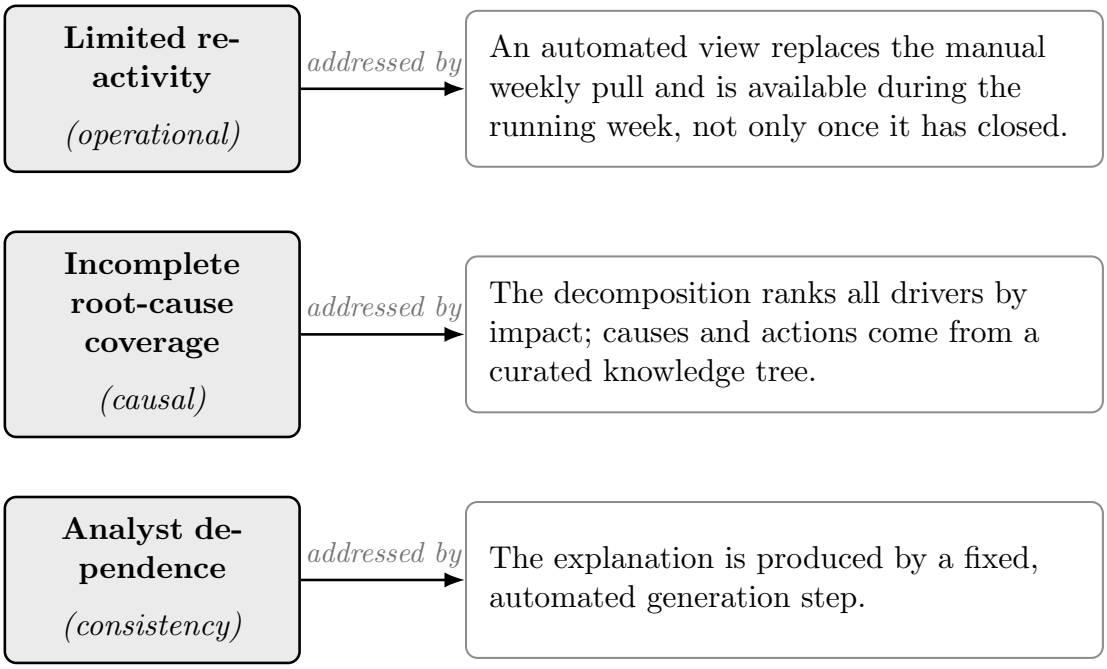

Figure 3.2: The three weaknesses of the manual baseline, the dimension each one touches, and how the rest of the thesis answers each one.

of invented causes. This causal weakness and the analyst dependence share the same root in the manual judgment of the manager, but they are not the same defect: one concerns whether the listed causes cover the whole gap and are ranked by impact, and the other is about whether the output survives the absence of a specific person. Reactivity stands apart, as the only operational one. The numbers, by contrast, were never a problem at all: the manager pastes them directly from the spreadsheet, and they turn into a risk only when processed by an LLM. How each of these is addressed is discussed in Chapter 4, where the three-contract framework is introduced.

3.4 Methodology

The study is organized around a single operation, developed following a longitudinal design: I worked on the system observing day after day the working routine.

The system presented in Chapter 4 was developed through an iterative loop: code, observe what the output is, isolate one issue at a time, fix it, validate the solution, and repeat; this loop ran roughly several dozen times, and each pass left a record of what had gone wrong and why. Those traces are the base material of the analysis: I organized them into families by the kind of error they represent, and labeled them as grounding failures in Chapter 4. The data behind the test sits on two layers. The development layer is the pilot site itself: three months of daily observation at DFV2, which is where failures came from. The evaluation layer is much wider, being a corpus of thirty under-target stations, each for a specific week; they were retrieved from the internal network, and paired with the explanations managers wrote for those same weeks. In Chapter 5 the framework is measured on two fronts: against the failure modes, and on how its generated narrative matches the human-written bridges.

Every metric in this thesis is anonymized: the headline indicator is denominated as Metric_R, its components Category_1 to Category_k. The values are synthetic in the illustrative examples and real in the operational data behind Chapter 5.

Chapter 4

The Three-Contract Framework

Chapter 3 closed by listing the three weaknesses and describing the iterative process followed during development. The summarized data a manager read after the week closed came too late; the listed causes were rarely complete or ranked by impact, and the depth of the analysis rode on the person in charge. This chapter presents the framework those weaknesses pushed me to build.

The answer takes its shape from the problem. Grounding has two dimensions, as introduced in Chapter 1: the numeric one, where a hard guarantee is possible, and the causal one, where the most that can be enforced is a bound. The narrative produced on this basis must hold on both sides. Literature on hallucinations verifies the claim against the source [30]; it does not enforce both simultaneously. The framework, instead, was built for exactly that.

4.1 Core Concept

In software engineering, a contract is the deal between a component and its caller: meet the precondition and the component guarantees its postcondition [34]. The core concept applies the same idea to a generation pipeline that, through three contracts, transforms a broad set of indicators into a complete narrative. None of them alone does the full job, because each one acts where the others cannot

reach; that is why a complete framework is needed. A pipeline of this shape offers exactly three points where a guarantee can attach: upstream where the numbers are computed; inside the prompt, on what the model is told; after the model, on what is produced. A fourth point would sit inside the generation itself, at the token level, where constrained decoding operates (Chapter 2); a frontier model served through an API gives no access to its decoder, so that point is out of reach here.

As presented in Figure 4.1, the first contract, *tile-as-source*, sits at the top of the pipeline. It is positioned at the beginning because it freezes the numbers before the model sees them, to take away the need for any LLM recomputation. The second, the *curated knowledge tree*, is positioned directly inside the prompt; it is used to instruct the model with causes and actions the domain admits. The last contract, the *sanitizer*, is located at the end of this pipeline; it serves as a deterministic check on the finished output to repair or refuse the narrow class of formal slips that could survive the first two.

Table 4.1 captures the key conceptual difference. Two of the three contracts bring a hard guarantee, the third a soft one instead. The two sides differ in what they constrain: a number either matches its source or it does not, and a deterministic check settles that with no judgment, so the numeric side admits a hard guarantee. For the causal side, instead, we cannot talk about hard guarantees because the tree bounds the set the model may pick from, but which cause represents the best fit to explain the movement is left to the LLM, so there is no deterministic wall to enforce it.

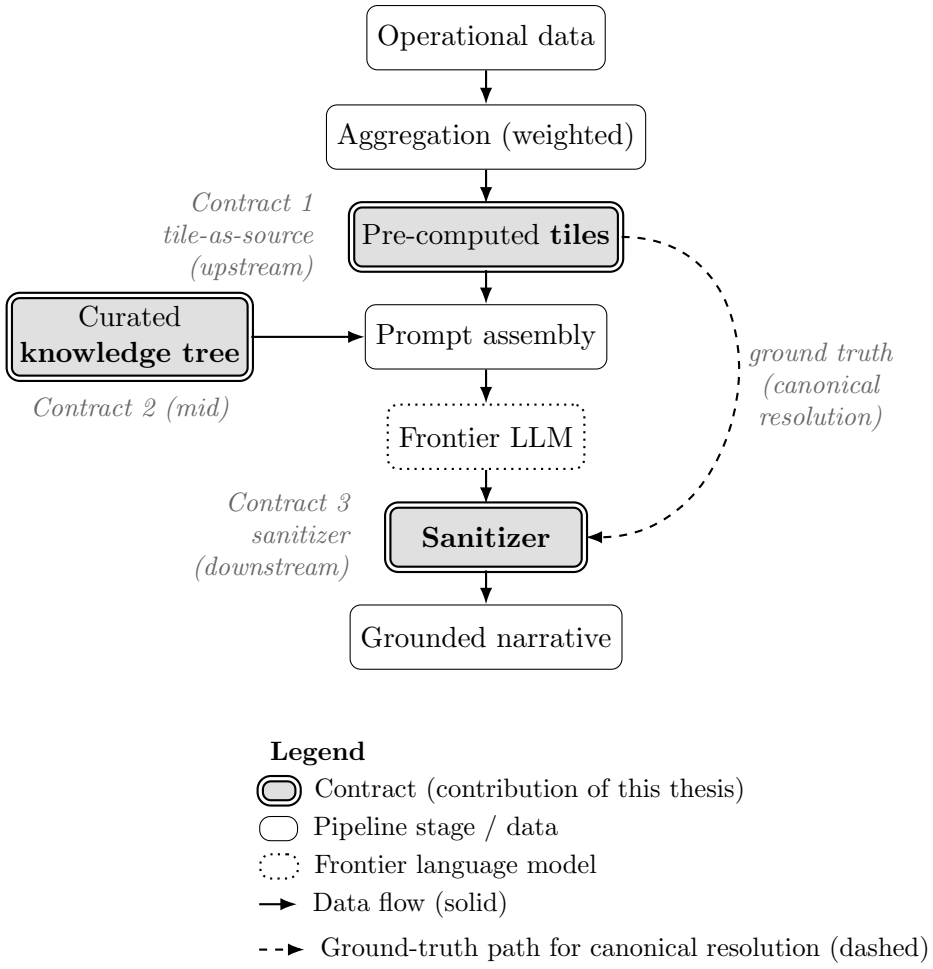

Figure 4.1: The AI-BI pipeline with the three contracts in place: tile-as-source freezes the numbers upstream, the knowledge tree bounds causes and actions inside the prompt, and the sanitizer audits the final narrative against the same tiles used to generate it.

	Tile-as-source	**Curated knowledge tree**	**Sanitizer**
Position	Upstream	Mid (prompt)	Downstream
Failure modes	Recalculation (F1) Rate-value hallucination (F2) Value-as-gap (F3) Precision drift (F6, partial)	Metric conflation (F4) Non-actionable recommendation (F5)	Precision drift (F6) Sign inversion (F7) Token concatenation (F8)
Guarantee	**Hard** prevent, on covered values	**Soft** bound	**Hard** repair or refuse

Table 4.1: The three contracts: where each acts on the pipeline, the failure modes it targets (detailed in Section 4.2), and the kind of guarantee it gives.

4.2 The failure catalog

Over the three months I spent building and watching the system, failures piled up into a small set of recurring modes. Every grounding failure observed falls into one of eight modes, and they sort into three classes based on what they break and where in the chain they can be caught. The classes refine the two requirements of Chapter 1: numeric and formal are the two halves of the numerical side, one about which value is reported and one about the form it arrives in, while the causal class matches the causal side. This distinction can be observed in Table 4.2, where the modes are listed with a short description of what happens when they occur.

Each class points at one of the three contracts this chapter presents. Tile-as-source (Section 4.3) defines what the model may report, and targets the numeric class; the knowledge tree (Section 4.4) bounds the admissible causes and actions, working on the causal side; the sanitizer (Section 4.5) checks the form of every value on the finished text, covering the formal class. Why one contract per class is a necessity and not a redundancy is the argument of Section 4.1, while how often each mode occurs, and how far the contracts push those counts down, is measured in Chapter 5.

Class	Mode	What happens
Numeric	F1	*Recalculation*: the model computes a value instead of copying the given one.
	F2	*Rate-value hallucination*: a rate that was never given is invented.
	F3	*Value-as-gap*: a contribution is stated as if it were the gap itself.
Causal	F4	*Metric conflation*: a domain term is given a wrong or invented meaning.
	F5	*Non-actionable recommendation*: an action outside the feasible set of the site is proposed.
Formal	F6	*Precision drift*: the canonical form is altered (49.0 shown as 49).
	F7	*Sign inversion*: a value flips its sign, or a comparative asserts the wrong direction.
	F8	*Token concatenation*: two metric labels fused into a non-existent one.

Table 4.2: The eight grounding failure modes observed during development, grouped by class.

The formal modes came first. Early builds of the pipeline carried no upstream contract, so numbers drifted, precision slipped, and the occasional comparative came out with the wrong sign, on real narratives. The tool that caught them, day after day, was a rudimentary deterministic check on the finished text, the seed of what Section 4.5 formalizes as the sanitizer. Then upstream contracts changed the picture: freezing the numbers before generation removed the opportunity for drifts or inversions, so the formal modes stopped showing up on their own. The next three sections present the contracts in pipeline order, upstream first, not in the order they were built.

4.3 Contract 1: tile-as-source

The first contract is founded on the idea that the safest number is the number that the model never computes. It is one single rule that might seem boring and

almost too plain to matter: every number that might appear in the narrative is only computed once; the LLM is never asked to calculate a percentage, a difference or anything that has not been deterministically precalculated. In contract terms: the *precondition* is that every number needed by the model exists as a frozen tile in the prompt; the *postcondition* is that every number in the narrative is a verbatim copy of a tile; the two boundaries at the end of this section state where this promise stops.

The idea is consistent with the one introduced in Chapter 2 for what regards semantic layers [13]. What is specific here is the target. A semantic layer manages the value inside a dashboard and stops there; instead I decided to push the same concept further onto the generated text, so what a manager reads is only the upstream computed tile, carried through unchanged.

Listing 4.1 shows how this rule is applied in the pipeline, through a frozen block of tiles that is directly injected into the model's prompt with a specific instruction to copy it verbatim. Each tile is a signed, pre-computed contribution, organized in a hierarchical order. It starts with the headline figure (`Metric_R`) that represents the overall metric, followed by the others (`Category_k`) that are decomposition tiles ranked by absolute movement, each already carrying its share of it.

Listing 4.1: Building the frozen-tiles block.

```
def tiles_block(c):
    L = [f"Metric_R OVERALL (frozen tiles - copy verbatim):",
         f"- Plan: {c['plan']:.3f}",
         f"- Actual: {c['actual']:.3f}",
         f"- Variance (Actual - Plan): {c['variance']:+.3f}",
         "",
         "DECOMPOSITION TILES (signed contribution, sum = Variance; ranked):"]
    tot = sum(abs(v) for v in c["buckets"].values())
    for k, v in material(c):                        # only |contribution| > 0.02
        pct = round(abs(v) / tot * 100) if tot else 0
        L.append(f"- Category_{k}: {v:+.3f} ({pct}% of absolute movement)")
    return "\n".join(L)
```

Table 4.1 assigns this contract a hard guarantee, and the reason is that copying carries most of the numeric load on its own: it takes away the need for any recalculation (F1), the room for invented rates (F2), the occasion to confuse values for deltas (F3), and helps with any precision drift since a copied value arrives with

the right decimals (F6).

There are two main boundaries for this contract. The first is that the rule binds only numbers that exist as tiles in the dashboard, not those the model may compute itself. The second one is deeper than the first, since it is true that it binds the number to the tile computed upstream, but there is no guarantee that the value represents the truth in the world. There is no check about the intrinsic correctness of the value itself: this is faithfulness, not factuality, as stated in Chapter 2.

4.4 Contract 2: curated knowledge tree

The tree used in the framework is a compact map, compiled from existing operational documentation, containing the causes each single driver admits together with feasible actions. Given that the aim of the tree is producing a single specific narrative, and not open question answering, the tree is small enough to maintain and be read in full: one entry per driver, a handful of admitted causes for each, and the actions paired with them. It is injected into the prompt as a list the model has to stay inside, and it is paired with the numeric rule so the narrative is held both on the numerical and causal side. In contract terms: the *precondition* is having a tree that carries the admitted causes and actions for the drivers; the *postcondition* is that every cause and action named in the narrative belongs to that set but, unlike the sanitizer, this cannot be deterministically guaranteed at generation time: a cause has no tile to be checked against.

At generation time the pipeline does not hand the whole tree to the model. Based on the drivers that caused the gap on the day, only a subset of the full causes together with the paired actions is selected and passed to the prompt. This is also the reason behind a language model in the loop: choosing which admitted causes best fit the week's gap is a judgment a fixed template cannot make. Listing 4.2 shows the scoped block as the LLM receives it: the complete set of admitted causes and actions for the drivers that moved. The decomposition those drivers come from reconciles to the total gap, so every material contribution is present and ranked by impact. This last point is the coverage weakness highlighted in

Chapter 3, solved by design instead of manual checks.

Listing 4.2: Building the scoped knowledge-tree block.

```
def tree_block(c):
    L = ["=== CURATED KNOWLEDGE TREE (use ONLY these causes/actions) ==="]
    for k, v in material(c):
        e = TREE[k]                                  # curated entry for this driver
        L.append(f"\nCategory_{k} (Stage: {e['Stage']}):")
        L.append("  Allowed root causes:")
        for rc in e["primary_root_causes"]:
            L.append(f"    - {rc}")
        L.append("  Allowed actions:")
        for a in e["immediate_actions"]:
            L.append(f"    - {a}")
    return "\n".join(L)
```

What the tree buys is the causal side of the narrative, the one that the other two contracts do not touch. With tile-as-source in place the model can still get the meaning of a metric wrong (F4), and the sanitizer does not guarantee that all actions produced are inside the feasible set (F5). The tree is what bounds both, closest in spirit to the methods that feed a small slice of a graph into the prompt [17]. Being a hand-curated tree allows for compactness, but it is also a major downside: the quality of it runs entirely on how fresh that curation stays; Chapter 6 returns to this point.

4.5 Contract 3: the sanitizer

This last deterministic contract runs directly on the finished text after the model. The first two shape what the LLM sees, while the sanitizer audits what the model produced, comparing every cited figure against its canonical value, when one exists, and repairs a small set of specific errors. In contract terms: the *precondition* is that for every metric in the narrative, a known-correct value of it exists; the *postcondition* is that every cited figure either matches the known value or is left visibly broken. It works through four mechanisms that run one after the other. The first is label disambiguation: before touching a figure, the sanitizer works out which metric that number belongs to, so it does not swap one driver's value for another's. Then canonical resolution checks that the value exists in the pipeline

at all, the ground-truth path drawn dashed in Figure 4.1. The magnitude guard comes third. It corrects a figure when the gap between the generated number and the dashboard one is small enough to look like a precision or recalculation drift, and refuses when that gap is too large to be one; the threshold is deliberately generous, half of the canonical value in Listing 4.3, because with every number frozen upstream a cited figure is almost always the right value in a slightly wrong form, so the guard only has to tell a drifted copy from a genuinely different one. Last comes the precision lock, that holds a value in its canonical shape, so a figure that should read one way does not appear in another. The magnitude guard acts on formal modes: it is what refuses a sign inversion (F7), since a flipped value lands far outside any plausible drift; a fused label (F8) instead fails canonical resolution, so it stays visibly broken.

Listing 4.3: Precision lock and magnitude guard.

```
def sanitize(text, canonical):
    for span in cited_numbers(text):            # every figure the model wrote
        m = canonical.get(span.metric)          # label disambiguation: which
            metric?
        if m is None:                           # canonical resolution: must
            exist
            continue
        if abs(span.value - m) <= abs(m) / 2:   # magnitude guard: small gap =
            drift
            text = replace(text, span, fmt(m))  # precision lock: canonical form
        # else: gap too large, leave the text untouched
    return text
```

Listing 4.3 shows the core of the numeric repair cited above and the choice to refuse the correction rather than force it. The reason for that comes directly from the operational setting: the worst result is a wrong sentence that reads as correct, worse than a visible error a reader could catch. An aggressive sanitizer that corrects any value close to the canonical one in a broken sentence would wash the break away, leaving a clean-looking lie. The conservative option I decided to go for preserves a big mismatch through the magnitude guard.

The sanitizer is a corrector rather than a preventer. It acts like a smoke detector that will probably stay off, as demonstrated in Chapter 5, but it is needed anyway: it is what enforces the numeric bound on every figure it can trace back to a tile.

This last contract is not free of risks. Label disambiguation can collide if two drivers share the same number: in that case the value could end up matched with the wrong label; or the cleanup of orphaned formatting around rate values can, if written too broadly, strip a figure that should have stayed. The risk is real but small. It is the price of having a downstream check that runs without judgment.

4.6 Prompt engineering

The contracts in the previous sections are not abstract ideas: the first two reach the model directly through the prompt while the last one stands after it on the finished text. The system prompt that drives the model is not designed to ask "in general terms" to be accurate. Chapter 2 already explained that a prompt is a request not a command, and better instructions raise the odds of a correct answer without making any errors impossible [6]. So this design does not try to overturn that verdict, it builds with it. The prompt is scoped to what a request can actually do: deliver the two pre-generation contracts (tile-as-source and curated knowledge tree) and cut the avoidable failures at the source. Specifically there is a directive for each failure mode observed during the development, nothing vaguer, so the LLM's efforts can be directed where needed.

The prompt has a well-defined structure composed by the directive block that sits on top, then the tiles block with the frozen values, the scoped tree block, and the skeleton of the narrative that must be followed. The skeleton also copies, for every action, the owner and the horizon the tree assigns it, so feasibility rides along with the action instead of being asked for. Listing 4.4 shows an excerpt of the directive block, with each directive tagged with the mode it answers. To be precise, in the production prompt, these base rules grow into more specific versions based on the domain they are referring to, but every variant traces back to one of the eight modes.

Listing 4.4: Excerpt of the directive block: one rule per observed failure mode.

```
1 RULES ( every failure mode answered):
2 - Copy every value from the TILES block verbatim; never
3   compute, round, or derive a number yourself.       # F1, F6
```

```
- Never state a rate or a share that is not given in
  the TILES block.                                  # F2, F3
- Name causes and actions ONLY from the KNOWLEDGE
  TREE block.                                       # F4, F5
- When comparing two values, state the direction
  explicitly and check it against the signs given.  # F7
- Use every metric label exactly as written; never
  merge two labels into one name.                   # F8
```

Given a competent but generic prompt, the one Chapter 2 warned about, a capable model recomputes numbers, invents the meaning of an unfamiliar acronym, and computes shares of the gap that nobody gave it. That is the naive baseline, failures at close to their full rate. The same model under the directive-per-failure prompt, with the tiles and tree blocks in place, stops doing all of it, down to the residue the tile contract already declared: a handful of totals the model sums on its own. The prompt is the connective tissue between contracts and the model. Chapter 5 takes this wiring apart step by step: each level of the ablation adds one contract together with its directives. The model is always the same; any difference between the levels comes only from the design itself.

4.7 Application

The three contracts run inside an application that the manager opens during the week to have a comprehensive view of the delivery station. It is the tool that answers the first weakness of Chapter 3: a lack of reactivity caused by pulling data manually, tile by tile, while the weekly report was still days away.

The application works by rebuilding the old spreadsheet weekly report with all its categories and subcategories, in a modern way by implementing charts and visual aids that guide the reader. One of the key parts, as anticipated, is the possibility to have an intra-week view: the tool shows, on any day of the running period, whether the station sits above or below its plan, so the picture no longer has to wait for the week to close. When a flagged week closes, one click automatically pulls back the full view of it, all the data plus some extra context to help the manager understand and control the metrics of their workplace. Having context

is a necessary condition because a manager reading the narrative must be capable of checking in real time what is being said without the need to search for the information by hand across different dashboards, the requirement Chapter 1 tied to trust. The bridge itself is produced by the engine of the previous sections, and because this generation step is fixed and identical for every station, the depth of the output no longer depends on who runs it, and that settles the writing side of the third weakness of Chapter 3; the judgment itself does not disappear, it moves upstream into the curation of the tree.

Figure 4.2 shows an anonymized sketch of the application, reusing the same synthetic values of Table 3.1. This tool is in the prototyping stage, so it is not used in daily production; in a huge company there are necessary steps that need to be taken and it is not a short process. What Chapter 5 measures, therefore, is not its adoption rate but the quality of what this application produces.

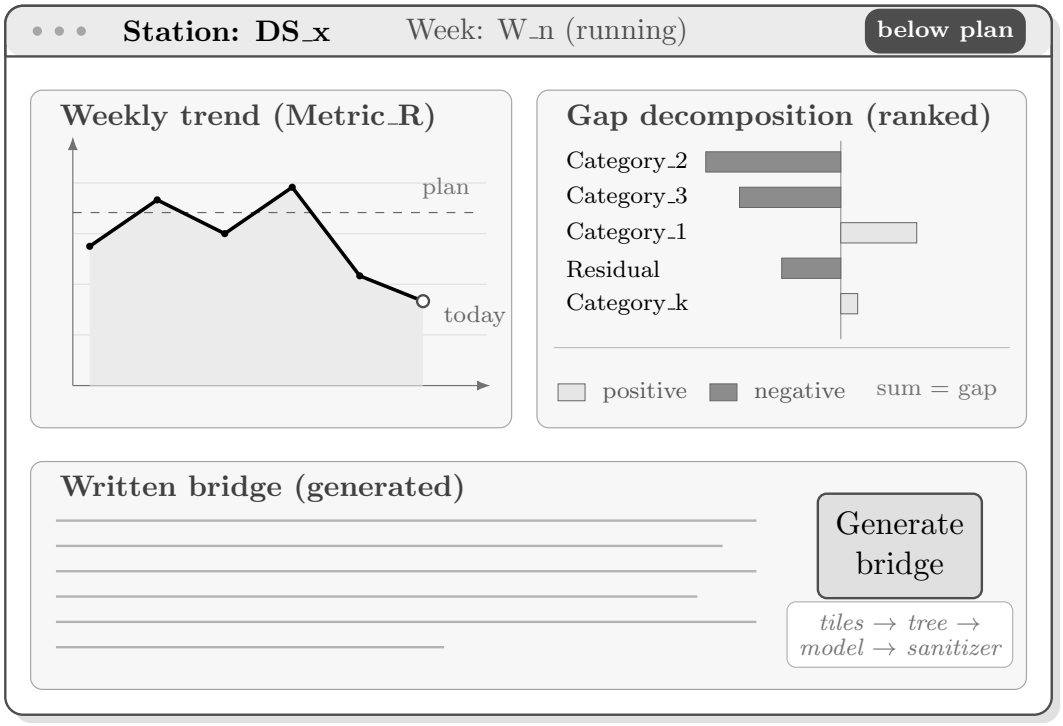

Figure 4.2: Anonymized wireframe of the application: the weekly trend against plan with the running-week position (left), the ranked signed decomposition of the closed week's gap (right, light bars positive and dark bars negative), and the generated bridge with the one-click step that runs the three contracts (bottom). Bar lengths are proportional to the synthetic values of Table 3.1.

Chapter 5

Results and Evaluation

Chapter 4 presented the three-contract framework and how it acts to ground the narrative. This chapter measures how that claim holds against thirty station-weeks of real data, each below target, each paired with the bridge a human manager wrote by hand for the same period. It takes the catalog of Section 4.2, collected over three months of observations, shows what ablation does to those failures, reports how close the final output sits to the expert's bridge, and closes on the limits of all of it.

5.1 Design of the evaluation

The evaluation answers two separate questions: whether the framework avoids the failure modes, and how close the generated narrative lands to the bridge a manager wrote. Each front is measured in its own way, and a language model takes part in only one of them; both run on the same corpus of narratives, generated along the ablation ladder of Figure 5.1.

The first front, which asks whether the framework reduces the failure modes, does not use any LLM judgment. For the numeric modes (F1, F2, F3) it relies on a deterministic check: every figure in the narrative has to trace back to a value the model was given. The causal modes (F4, F5) are counted by hand, by a single

rater, against predefined rules: the named causes and actions have to exist in the admitted domain. Finally, the formal modes (F6, F7, F8) are tested by injection, since the final pipeline no longer produces them.

The second front compares each generated narrative with the corresponding human bridge. It uses real data from thirty different delivery stations, each under target during a specific week; each station is run through the four levels of the ladder, which gives one hundred and twenty narratives. All the stations belong to the same country network, span two building types, and cover gap magnitudes from small to large. The production model is a frontier language model, the same one the app runs, and the prompt is the full app prompt, not a reduced version. Each generation is isolated and stateless: one conversation per station per level, three concurrent workers, and no contamination across stations.

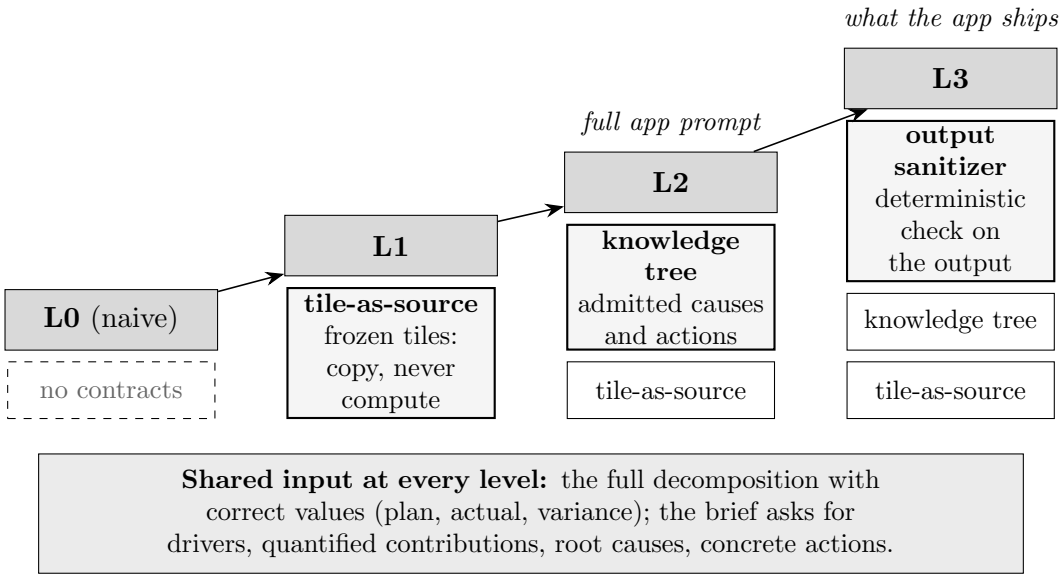

Figure 5.1: The ablation ladder used in the evaluation. All four levels receive the same input (bottom bar); each level stacks one more contract on top of the previous one, and the block with the thick border is the contract that enters at that step. L2 is the full app prompt, L3 the configuration the app ships.

Figure 5.1 shows that the ladder itself has four rungs, each adding a contract. The bottom bar is the reason why the naive level is not a strawman: L0 already hands the model the full decomposition with the correct values, plan, actual, and variance

included, and already asks the model to name drivers, quantify the contribution, and give causes and actions. This first level shows what a capable model does when nothing constrains it. A contract is then added by each of the rungs above: L1 freezes the values into tiles, L2 (the full app prompt) bounds causes and actions, and L3 (the configuration the app ships) places the sanitizer downstream of the generation.

5.2 First front: the failure modes

This section organizes the failure modes in the order the contracts enter the pipeline. Figure 5.2 collects the counts. Each different mode is counted with the procedure that fits its nature, as set out in Section 5.1, and the totals are reported across the thirty station-weeks.

	L0 *baseline*	**L1** *tile-as-source*	**L2** *tree*	**L3** *sanitizer*
F1 recalculation	44	1	5	5
F2 rate-value hallucination	21	0	0	0
F3 value-as-gap	22	0	0	0
F4 metric conflation	11	6	0	0
F5 non-actionable recommendation	31	26	0	0

Figure 5.2: Failure-mode counts across the ablation ladder, as total instances over the 30 station-weeks.

Starting with the numeric modes, at L0 the model fabricates numbers freely: it recalculates forty-four numbers (F1), invents twenty-one rates (F2), and states twenty-two values-as-gap (F3). One rung of the ladder is enough to remove nearly all of it. In particular, the increase observed for F1 at levels L2 and L3 is not a regression of the tile contract: the tree makes the narrative richer, more buckets get discussed together, and the model finds more occasions to state their total. The residual five are all such self-summed totals, the boundary declared in Section 4.3.

The causal modes (F4, F5) shrink at L1 and disappear completely only when the tree enters at level L2. At the previous two levels the model invents definitions for the different buckets, reading operational meaning into labels it was never given, and proposes out-of-mandate actions; both disappear at L2.

For what regards the formal modes (F6, F7, F8), they cannot be read off the natural output: by the time the sanitizer runs, the upstream contracts have already removed almost everything it is built to catch, and counting occurrences would report a zero that says nothing about the guard itself. On the natural runs it indeed stayed idle: zero corrections across the thirty shipped narratives. So the sanitizer is measured by injection: the four numeric faults are each planted on all sixty-six sites of the shipped narratives, two hundred and sixty-four injections in total, plus a separate sweep that fuses the label of one metric with the value of another on the same sixty-six sites. Figure 5.3 shows the outcome of that stress test, which the sanitizer fully passed, with zero false repairs.

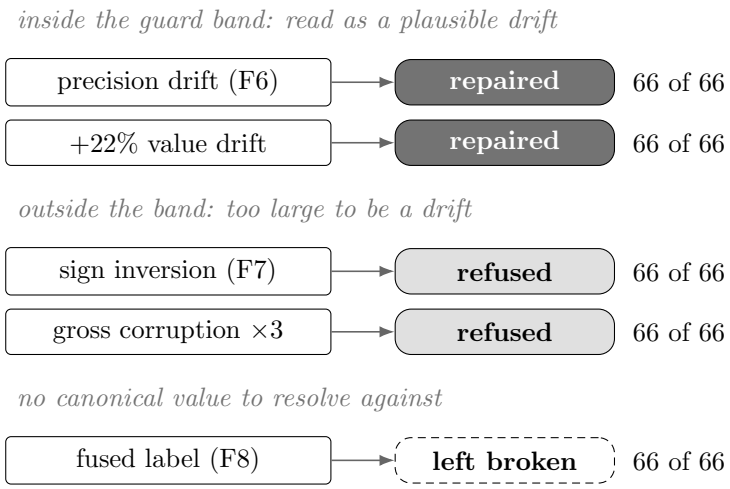

Figure 5.3: The sanitizer stress test: each fault planted on the 66 candidate sites of the shipped narratives. Dark chips are repairs to the canonical value, light chips are refusals that leave the fault visible, and the dashed chip is a fault with no canonical resolution, left untouched. Every outcome is 66 of 66: zero false repairs.

Chapter 6

Discussion and Future Directions

6.1 Synthesis

How the observed results successfully answer the initial research question.

6.2 Contributions

The specific value added to the existing literature regarding LLMs and BI.

6.3 Generalizability

An assessment of how well the three-contract framework can be adapted and applied to other contexts or industries.

6.4 Limitations

A transparent look at the limits of the current thesis work.

6.5 Future Developments

Potential next steps, such as extending the framework to support interactive Q&A functionalities based on the same underlying schema.

Bibliography

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Curran Associates, Inc., 2020.

[2] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, Ho Shu Chan, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[3] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[4] Ankur P. Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. Totto: A controlled table-to-text generation dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1173–1186. Association for Computational Linguistics, 2020.

[5] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for*

Computational Linguistics (EACL): System Demonstrations, pages 150–158. Association for Computational Linguistics, 2024.

[6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837. Curran Associates, Inc., 2022.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[9] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates, Inc., 2023.

[10] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023.

[11] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023.

[12] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models.

In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9895–9901. Association for Computational Linguistics, 2021.

[13] dbt Labs. dbt semantic layer and metricflow. `https://docs.getdbt.com/docs/build/about-metricflow`, 2024.

[14] Cube Dev. Cube: the semantic layer for data apps. `https://cube.dev/`, 2024.

[15] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[16] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.

[17] Jinheon Baek, Alham Fikri Aji, and Amir Saffari. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. In *Proceedings of the First Workshop on Matching From Unstructured and Structured Data (MATCHING 2023)*, pages 70–98. Association for Computational Linguistics, 2023.

[18] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921. Association for Computational Linguistics, 2018.

[19] Leixian Shen, Enya Shen, Yuyu Luo, Xiaocong Yang, Xuming Hu, Xiongshuai Zhang, Zhiwei Tai, and Jianmin Wang. Towards natural language interfaces for data visualization: A survey. *IEEE Transactions on Visualization and Computer Graphics*, 29(6):3121–3144, 2023.

[20] Yizhang Zhu, Shiyin Du, Boyan Li, Yuyu Luo, and Nan Tang. Are large language models good statisticians? In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Curran Associates, Inc., 2024.

[21] Jeho Choi. Fact-consistency evaluation of text-to-sql generation for business intelligence using exaone 3.5. *arXiv preprint arXiv:2505.00060*, 2025.

[22] Salesforce Tableau. Tableau pulse: Ai-generated metric insights. `https://www.tableau.com/products/tableau-pulse`, 2024.

[23] Microsoft. Copilot in power bi. `https://learn.microsoft.com/power-bi/create-reports/copilot-introduction`, 2024.

[24] Salesforce Tableau. The insights platform and insight types in Tableau Pulse. `https://help.tableau.com/current/online/en-us/pulse_insights_platform_insight_types.htm`, 2024.

[25] Salesforce AI Research. Developing reliable LLM-powered insight summarization for Tableau Pulse. `https://www.salesforce.com/blog/tableau-pulse/`, 2024.

[26] Ran Zhang, Mohannad Elhamod, et al. Data-to-dashboard: Multi-agent llm framework for insightful visualization in enterprise analytics. *arXiv preprint arXiv:2505.23695*, 2025.

[27] Hejing Chen, Yixuan Lu, Yijing Wei, Jinyan Lyu, Ruibo Wu, and Chen Chen. Causal-llm: A hybrid framework for automated budgetary variance diagnosis and reasoning. In *Proceedings of the 2026 International Conference on Big Data and Internet of Things and Cloud Networks (BDICN)*. Association for Computing Machinery, 2026.

[28] Matteo Francia, Enrico Gallinucci, Matteo Golfarelli, and Stefano Rizzi. Vool: A modular insight-based framework for vocalizing olap sessions. *Information Systems*, 129:102496, 2025.

[29] Xiufeng Liu and Yanyan Yang. Llm-assisted conversational navigation over multidimensional data cubes. *Expert Systems with Applications*, 328:132958, 2026.

[30] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12076–12100. Association for Computational Linguistics, 2023.

[31] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. Halueval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6449–6464. Association for Computational Linguistics, 2023.

[32] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3214–3252. Association for Computational Linguistics, 2022.

[33] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9004–9017. Association for Computational Linguistics, 2023.

[34] Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992.

Acknowledgements

Optional. Max 1 page.